

# 1. Product Requirements Document — PRD

## 1.1 Product Name

### AatmaMirror + DharmaMind AI

#### Tagline

**Pause. Reflect. Choose wiser tomorrows.**

---

## 1.2 Product Vision

Build a **reflective AI mental wellness platform** that helps students share emotional problems and receive **balanced, safe, scripture-grounded, reflection-first guidance**.

The product should not behave like a normal chatbot that simply validates the user. It should help the user:

- Pause before reacting
  - Reflect on consequences
  - Understand emotions
  - Receive Bhagavad Gita-grounded guidance
  - Get crisis-safe support when needed
-

## **1.3 Problem Statement**

Students face rising mental health challenges such as stress, anxiety, depression, loneliness, exam pressure, relationship conflicts, and impulsive decision-making.

Existing AI tools often provide:

- Instant emotional validation
- One-sided support
- Generic advice
- No ethical reflection
- No crisis escalation
- No culturally grounded wisdom

Your PPT already highlights this clearly: current AI lacks ethical reasoning, may encourage impulsive decisions, and gives reactive comfort instead of guided reflection.

---

## **1.4 Target Users**

### **Primary Users**

- College students
- Youth facing stress, confusion, loneliness, or emotional conflict

### **Secondary Users**

- College counsellors

- Wellness teams
- Student support cells

#### **Future Users**

- Working professionals
  - Families
  - Institutions
  - Spiritual wellness communities
- 

### **1.5 Core User Problems**

| <b>Problem</b>             | <b>User Pain</b>                      |
|----------------------------|---------------------------------------|
| Emotional overload         | User wants instant support            |
| Impulsive decisions        | User may act without reflection       |
| AI over-validation         | Chatbots may support only user's side |
| Lack of cultural grounding | Advice feels generic                  |
| Counsellor overload        | Human support is delayed              |

Crisis risk

Severe distress may  
go unnoticed

---

## **1.6 Product Goals for 3-Day MVP**

By the end of 3 days, build a working prototype that demonstrates:

1. User can enter a mental wellness query
  2. System detects emotion/risk level
  3. System intentionally introduces a reflection pause
  4. System retrieves relevant Bhagavad Gita wisdom using RAG
  5. System generates safe, balanced guidance
  6. System shows verse/reference-backed response
  7. System escalates if crisis keywords are detected
  8. Admin can generate awareness content from common problems
- 

## **1.7 Non-Goals for 3-Day MVP**

Do **not** build these in 3 days:

- Full mobile app
- Real therapist dashboard
- Payment/subscription

- Full authentication with roles
- Real-time voice conversation
- Multi-scripture support
- Production-grade crisis intervention
- Wearable integration
- Fine-tuned LLM

These can be future scope.

---

## **1.8 MVP Scope**

### **Must-Have**

- Landing/dashboard screen
- Chat interface
- Emotion detection
- Risk classification
- RAG retrieval from Bhagavad Gita dataset
- AI-generated reflective answer
- Constitutional safety rules
- Crisis fallback response
- Admin content generator
- Demo-ready UI

### **Should-Have**

- Query category selection
- Verse references
- Future-self reflection prompt

- 5 free query counter
- Simple user history

#### Could-Have

- Voice input
  - Text-to-speech
  - Mood graph
  - Admin analytics
- 

## 1.9 Key Features

Your PDF already lists the main features on page 4. We will convert them into implementable MVP modules.

### Feature 1: PausePulse

**Purpose:** Prevent instant reactive advice.

#### MVP Behavior

Before answer generation, show:

“Take 5 seconds. What outcome do you truly want from this situation?”

#### Implementation

- Frontend delay: 3–5 seconds
- Show reflection prompt
- Then generate AI response

---

## **Feature 2: ForwardMirror**

**Purpose:** Help user see future consequences.

### **MVP Behavior**

AI response includes:

- If you react impulsively
- If you respond calmly
- Better next step

### **Example**

If you react now: conflict may increase.

If you pause: you may respond with clarity.

Suggested step: write your feelings before messaging them.

---

## **Feature 3: EthosGuard**

**Purpose:** Constitutional AI safety layer.

### **MVP Rules**

- Do not blindly agree with user
- Do not give medical/legal/financial certainty
- Do not shame user
- Do not claim to be God/guru
- Encourage reflection

- Use calm language
  - Escalate crisis cases
- 

## **Feature 4: Scripture RAG Engine**

**Purpose:** Retrieve relevant Bhagavad Gita wisdom.

**MVP Dataset**

Start with 25–40 curated verses around:

- Stress
  - Duty
  - Detachment
  - Anger
  - Desire
  - Fear
  - Confusion
  - Discipline
  - Self-control
  - Failure
  - Relationships
- 

## **Feature 5: Crisis Escalator**

**Purpose:** Detect high-risk messages.

**MVP Crisis Keywords**

- “suicide”
- “kill myself”
- “end my life”
- “don’t want to live”
- “self harm”
- “hurt myself”

#### **MVP Response**

- Do not generate spiritual advice only
  - Show urgent support message
  - Ask user to contact trusted person/emergency helpline
  - Optionally show counsellor escalation UI
- 

### **Feature 6: Emotional Risk Detection**

**Purpose:** Classify emotional tone.

#### **MVP Emotion Labels**

- Stress
- Sadness
- Anger
- Fear
- Confusion
- Relationship pain
- Career pressure
- Crisis

Can be done using:

- Rule-based keywords for hackathon
  - LLM classification if time permits
- 

## **1.10 User Stories**

### **Student User**

As a student, I want to share my emotional problem so that I can receive calm and balanced guidance.

As a student, I want the AI to help me reflect before reacting so that I do not make impulsive decisions.

As a student, I want Gita-based wisdom in simple language so that I can apply it to my real life.

---

### **Admin User**

As an admin, I want to generate wellness content from common problems so that I can spread awareness through reels and blogs.

As an admin, I want to see common emotional categories so that I understand student wellness trends.

---

## **1.11 Success Metrics for Hackathon**

### **Demo Metrics**

- Chat response works end-to-end

- Verse-backed answer appears
- Emotion/risk label appears
- Crisis case triggers safe fallback
- Admin generates reel/blog content
- UI looks polished

#### **Product Metrics Later**

- Daily active users
  - Average sessions per user
  - Query-to-reflection completion rate
  - Crisis detection accuracy
  - Subscription conversion
  - Content engagement
- 

## **2. High-Level Design — HLD**

### **2.1 System Overview**

User / Admin



Frontend App



Backend API



AI Orchestrator



Emotion Detection + Safety Filter



RAG Retrieval Engine



LLM Response Generator



EthosGuard Validator



Final Response

---

## 2.2 Main Components

### 1. Frontend

#### Pages

- Landing page
- User chat page
- Admin dashboard
- Content generator page
- Demo analytics page

#### Responsibilities

- Capture user query
- Show PausePulse reflection
- Display AI response
- Display verse references
- Display risk label

- Show crisis warning if needed
  - Admin content generation UI
- 

## **2. Backend API**

### **Responsibilities**

- Receive chat request
  - Run emotion/risk detection
  - Retrieve relevant verses
  - Call LLM
  - Validate response
  - Save chat logs
  - Manage query count
  - Generate admin content
- 

## **3. AI Orchestrator**

This is the brain of the app.

### **Responsibilities**

- Classify user query
- Decide whether it is safe
- Retrieve Gita context
- Build final prompt
- Generate response

- Apply constitutional rules
- 

## **4. RAG Engine**

### **Responsibilities**

- Store Bhagavad Gita chunks
  - Generate embeddings
  - Search similar chunks
  - Return top relevant verses
- 

## **5. EthosGuard Safety Layer**

### **Responsibilities**

- Check crisis risk
  - Prevent harmful advice
  - Prevent over-validation
  - Ensure balanced response
  - Ensure answer is grounded
- 

## **6. Admin Content Engine**

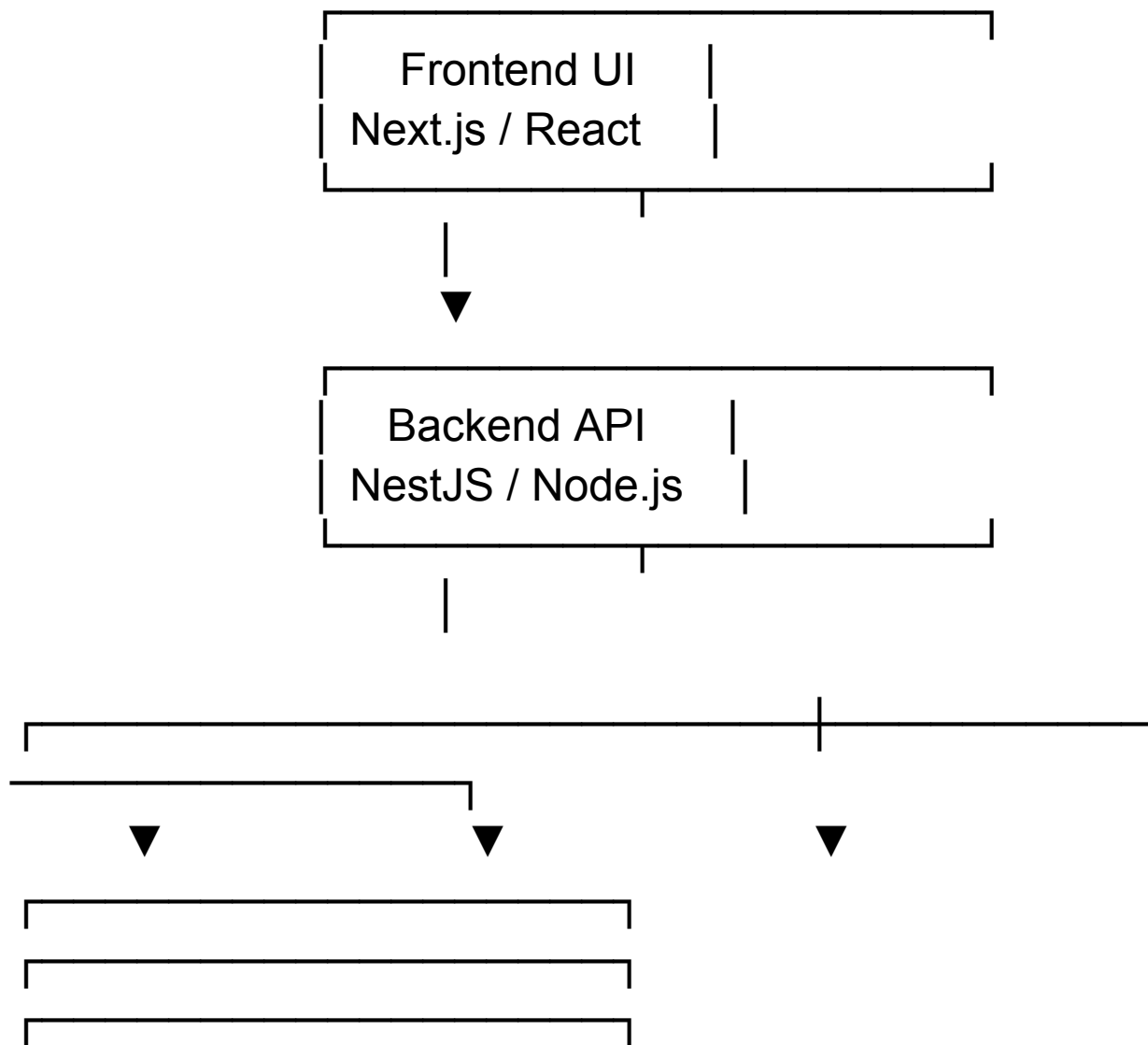
### **Responsibilities**

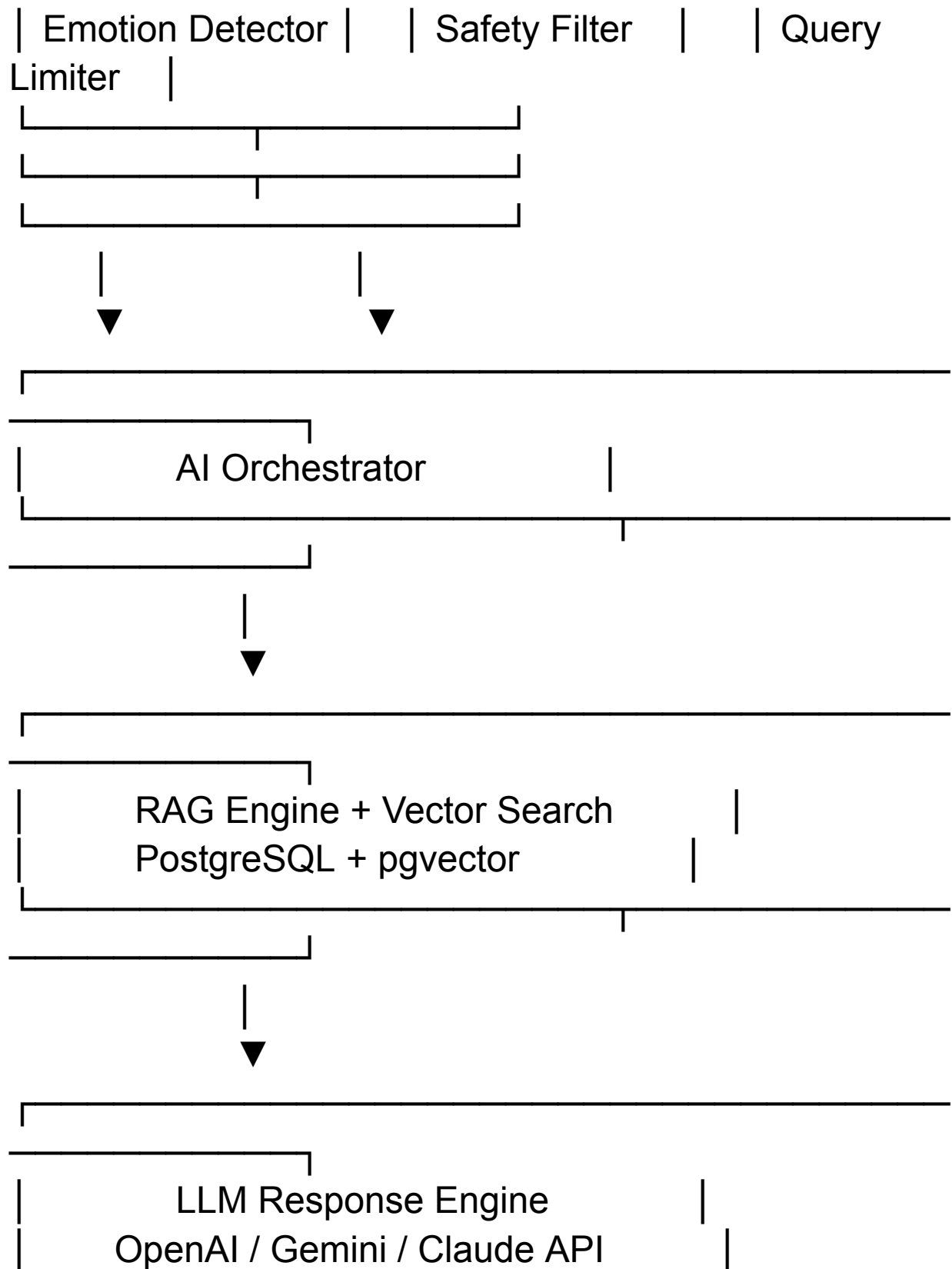
- Generate:
  - Blog

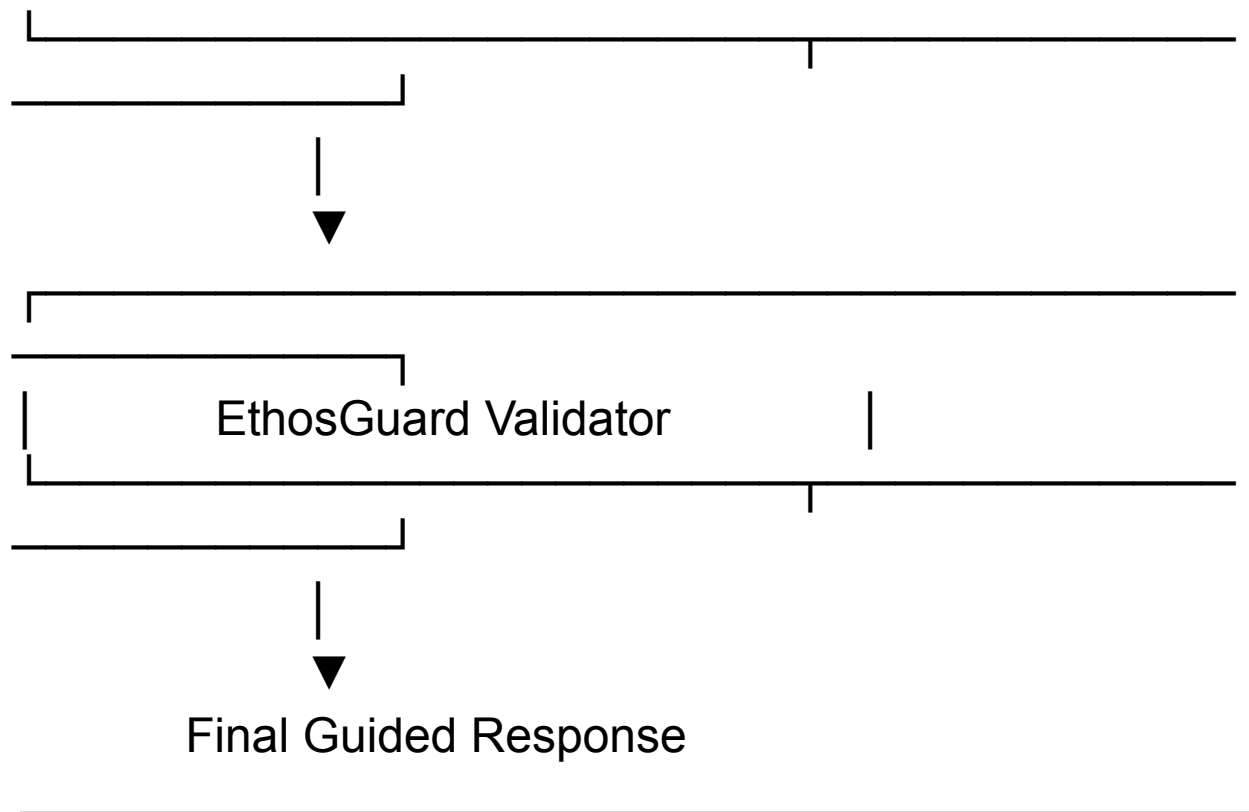
- Reel script
- Instagram caption
- Image prompt
- YouTube short script

---

## 2.3 HLD Architecture Diagram







## 2.4 Recommended 3-Day Tech Stack

Since this is a hackathon, use the fastest stack.

### Frontend

Next.js + TypeScript + Tailwind CSS

### Backend

NestJS OR Next.js API routes

For 3 days, I recommend:

Next.js full-stack app

Why?

- Faster setup
  - Frontend + backend in one repo
  - Easier deployment on Vercel
  - Less boilerplate than separate NestJS backend
- 

## **Database**

For hackathon MVP:

**Option A — Fastest**

JSON file dataset + in-memory similarity

**Option B — Better Demo**

Supabase PostgreSQL + pgvector

**My Recommendation**

Use:

Supabase + pgvector

If time becomes tight, fallback to JSON-based retrieval.

---

## **AI**

OpenAI / Gemini API

For cost and speed:

- Gemini Flash

- GPT-4o-mini
  - Claude Haiku
- 

## **Deployment**

Vercel for frontend/backend

Supabase for DB

---

# **3. Low-Level Design — LLD**

## **3.1 Data Models**

### **User**

```
type User = {  
  id: string;  
  name?: string;  
  email?: string;  
  queryCount: number;  
  plan: "FREE" | "PREMIUM";  
  createdAt: Date;  
};
```

---

### **ChatMessage**

```
type ChatMessage = {  
  id: string;
```

```
userId: string;  
message: string;  
response: string;  
emotion: string;  
riskLevel: "LOW" | "MEDIUM" | "HIGH" | "CRISIS";  
category: string;  
versesUsed: string[];  
createdAt: Date;  
};
```

---

### **GitaVerse**

```
type GitaVerse = {  
  id: string;  
  chapter: number;  
  verse: number;  
  sanskrit?: string;  
  translation: string;  
  meaning: string;  
  tags: string[];  
  category: string[];  
  embedding?: number[];  
};
```

---

### **GeneratedContent**

```
type GeneratedContent = {
```

```
id: string;
topic: string;
type: "BLOG" | "REEL" | "CAPTION" |
"IMAGE_PROMPT";
content: string;
createdAt: Date;
};
```

---

## 3.2 Database Tables

### **users**

```
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT,
  email TEXT,
  query_count INT DEFAULT 0,
  plan TEXT DEFAULT 'FREE',
  created_at TIMESTAMP DEFAULT NOW()
);
```

---

### **gita\_verses**

```
CREATE TABLE gita_verses (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  chapter INT NOT NULL,
  verse INT NOT NULL,
```

```
translation TEXT NOT NULL,  
meaning TEXT NOT NULL,  
tags TEXT[],  
category TEXT[],  
embedding VECTOR(1536),  
created_at TIMESTAMP DEFAULT NOW()  
);
```

---

### **chat\_messages**

```
CREATE TABLE chat_messages (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID,  
  message TEXT NOT NULL,  
  response TEXT NOT NULL,  
  emotion TEXT,  
  risk_level TEXT,  
  category TEXT,  
  verses_used JSONB,  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

---

### **generated\_content**

```
CREATE TABLE generated_content (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  topic TEXT,
```

```
type TEXT,  
content TEXT,  
created_at TIMESTAMP DEFAULT NOW()  
);
```

---

### 3.3 API Design

#### Chat API

**POST** /api/chat

##### Request

```
{  
  "userId": "demo-user",  
  "message": "I am stressed about exams and feel like I will  
fail",  
  "category": "exam_stress"  
}
```

##### Response

```
{  
  "emotion": "stress",  
  "riskLevel": "LOW",  
  "pausePrompt": "Before reacting, take a moment and ask:  
what is one action in your control?",  
  "answer": "Inspired by the Bhagavad Gita...",  
  "verses": [  
    {
```

```
    "chapter": 2,  
    "verse": 47,  
    "translation": "You have a right to perform your duty..."  
  }  
],  
  "forwardMirror": {  
    "impulsivePath": "Panic may reduce your focus.",  
    "reflectivePath": "Calm planning can improve your next  
step."  
  }  
}
```

---

## Admin Content API

**POST** `/api/admin/generate-content`

### Request

```
{  
  "topic": "Exam stress",  
  "type": "REEL"  
}
```

### Response

```
{  
  "title": "When exam pressure feels heavy",  
  "script": "...",  
  "caption": "...",  
}
```

```
"hashtags": ["#StudentWellness", "#BhagavadGita",  
"#MentalHealth"]  
}
```

---

## Verse Search API

**POST** /api/rag/search

### Request

```
{  
  "query": "I am angry and unable to control myself"  
}
```

### Response

```
{  
  "verses": [  
    {  
      "chapter": 2,  
      "verse": 63,  
      "meaning": "Anger clouds judgment..."  
    }  
  ]  
}
```

---

## 3.4 AI Pipeline LLD

### Step 1: Receive User Query

Input: user message + category

---

## Step 2: Emotion Detection

For MVP, use hybrid rule-based + LLM.

### Rule-Based Example

```
const emotionKeywords = {  
  stress: ["stress", "pressure", "exam", "deadline"],  
  anger: ["angry", "rage", "irritated"],  
  sadness: ["sad", "lonely", "crying"],  
  fear: ["scared", "afraid", "anxious"],  
  crisis: ["suicide", "kill myself", "end my life"]  
};
```

---

## Step 3: Risk Classification

```
if (containsCrisisKeywords(message)) {  
  riskLevel = "CRISIS";  
} else if (containsSevereDistress(message)) {  
  riskLevel = "HIGH";  
} else {  
  riskLevel = "LOW";  
}
```

---

## Step 4: Crisis Escalation

If crisis:

Return safe crisis response.

Do not continue normal Gita advice generation.

---

## **Step 5: RAG Retrieval**

**Query**

User message + detected emotion + category

**Retrieve**

Top 3–5 verses from Gita dataset.

---

## **Step 6: Prompt Construction**

You are AatmaMirror, a reflective AI wellness guide.

**Rules:**

- Do not blindly validate the user.
- Encourage pause, reflection, and wise action.
- Use only the provided Bhagavad Gita context.
- Do not invent verse references.
- Do not give medical diagnosis.
- If crisis is detected, recommend immediate human help.
- Keep response short, calm, and practical.

**User Problem:**

{user\_message}

Emotion:

{emotion}

Relevant Gita Context:

{retrieved\_verses}

Generate:

1. Gentle acknowledgement
  2. Reflection question
  3. Gita-based insight
  4. Practical next step
  5. Future-self mirror
- 

## **Step 7: Response Validation**

Run a lightweight validation:

Check:

- Is it safe?
  - Is it balanced?
  - Is it not over-validating?
  - Are verses from retrieved context only?
  - Is it short and clear?
-

## Step 8: Save Chat

Store:

- User query
  - Emotion
  - Risk
  - Response
  - Verses used
- 

## 4. Three-Day Execution Plan

This is the most important part.

### Overall Strategy

Do not try to build everything.

Build a **demo-first MVP**.

Your final demo should show:

Student enters emotional problem

- AI detects emotion
- PausePulse appears
- Gita verse is retrieved
- Balanced response appears
- Admin generates reel/blog
- Crisis query triggers escalation

That is enough to impress judges.

---

## Day 0 — Before Coding: 2–3 Hours

### Research & Setup

#### Goal

Finalize scope, dataset, repo, design, and demo story.

#### Tasks

##### Product

- Finalize product name: **AatmaMirror + DharmaMind AI**
- Finalize target user: **students**
- Finalize 5 demo categories:
  - Exam stress
  - Career confusion
  - Relationship conflict
  - Anger
  - Loneliness

##### Research

- Select 25–40 Bhagavad Gita verses
- Map each verse to tags
- Prepare dataset JSON

## Engineering

- Create GitHub repo
- Create Next.js app
- Setup Tailwind
- Setup environment variables
- Create basic UI layout

## Deliverables

- Final feature scope
  - Dataset JSON
  - Repo ready
  - UI wireframe ready
- 

# Day 1 — Core Product + UI + Dataset

## Goal

Build the frontend screens and basic backend flow.

---

## Morning: UI Foundation

### Build Pages

/

/chat

/admin

/demo

### Components

ChatBox

MessageBubble

EmotionBadge

RiskBadge

VerseCard

PausePulseCard

ForwardMirrorCard

AdminContentForm

---

### Afternoon: Dataset + Mock RAG

Create:

/data/gita-verses.json

Example:

```
{
  "id": "gita-2-47",
  "chapter": 2,
  "verse": 47,
  "translation": "You have a right to perform your duty...",
  "meaning": "Focus on effort, not anxiety over results.",
  "tags": ["stress", "career", "exam", "detachment"],
  "category": ["student", "career", "stress"]
}
```

---

## Evening: Basic Chat API

Create:

POST /api/chat

For Day 1, retrieval can be keyword-based:

message contains "exam" → retrieve exam/stress verses

message contains "angry" → retrieve anger verses

### Day 1 Deliverables

- Working chat UI
- Basic API connected
- Dataset loaded
- Keyword-based verse retrieval
- Basic response shown

---

## Day 2 — AI Integration + RAG + Safety

### Goal

Make the product intelligent and demo-worthy.

---

### Morning: Emotion Detection

Implement:

```
detectEmotion(message)
detectRisk(message)
detectCategory(message)
```

Output

```
{
  "emotion": "stress",
  "riskLevel": "LOW",
  "category": "exam_stress"
}
```

---

## **Midday: LLM Response Generation**

Integrate OpenAI/Gemini.

Create:

```
/lib/ai/generateReflectiveResponse.ts
```

Response structure:

```
{
  "acknowledgement": "",
  "reflectionQuestion": "",
  "gitaInsight": "",
  "practicalStep": "",
  "forwardMirror": "",
  "verses": []
}
```

}

---

## **Afternoon: EthosGuard**

Create constitution rules:

/lib/ai/constitution.ts

Rules:

- No blind validation
  - No harmful advice
  - No fake verse references
  - No medical diagnosis
  - Reflection-first tone
  - Crisis-safe fallback
- 

## **Evening: Crisis Escalator**

Test with:

I want to end my life

Expected output:

- No normal AI answer
- Crisis support response
- “Please contact trusted person / emergency help immediately”

- Option to show helpline placeholder

## **Day 2 Deliverables**

- Emotion detection working
  - Risk detection working
  - AI-generated answer working
  - Verse-backed response working
  - Crisis fallback working
- 

## **Day 3 — Admin Panel + Polish + Deployment**

### **Goal**

Finish demo, deploy, prepare pitch.

---

### **Morning: Admin Content Generator**

Build:

/admin

Admin enters:

Topic: Exam stress

Content Type: Reel

AI generates:

- Reel hook
  - 30-sec script
  - Caption
  - Hashtags
  - Image prompt
- 

## **Afternoon: Demo Flow Polish**

Add:

- Loading states
  - PausePulse delay animation
  - Emotion badge
  - Verse card
  - ForwardMirror card
  - Crisis alert UI
  - Query count badge
- 

## **Evening: Deployment**

Deploy:

Vercel

Prepare:

- Live demo URL
- Backup local demo
- 2-min screen recording
- PPT final screenshots

### **Day 3 Deliverables**

- Deployed app
  - Working user chat
  - Working admin content generator
  - Crisis demo
  - Final pitch flow
  - Screenshots for PPT
- 

## **5. Detailed Task Breakdown**

### **Frontend Tasks**

| <b>Task</b>     | <b>Prio<br/>rity</b> |
|-----------------|----------------------|
| Landing<br>page | P1                   |
| Chat UI         | P0                   |
| Admin page      | P0                   |

|                      |    |
|----------------------|----|
| Emotion badge        | P1 |
| Verse card           | P0 |
| PausePulse animation | P1 |
| Crisis alert UI      | P0 |
| Query counter        | P2 |
| Responsive design    | P1 |

---

## Backend Tasks

| Task                        | Priority |
|-----------------------------|----------|
| /api/chat                   | P0       |
| /api/admin/generate-content | P0       |
| Emotion detection           | P0       |

|                 |    |
|-----------------|----|
| Risk detection  | P0 |
| Verse retrieval | P0 |
| LLM call        | P0 |
| Crisis fallback | P0 |
| Save chat logs  | P2 |

---

## AI Tasks

| Task                    | Priority |
|-------------------------|----------|
| Gita dataset            | P0       |
| Prompt<br>template      | P0       |
| Constitution<br>rules   | P0       |
| Response<br>JSON schema | P0       |
| Admin content<br>prompt | P1       |
| Validation<br>prompt    | P2       |

Embeddings/v P2 for  
ector DB hackatho  
n

---

## 6. Recommended Folder Structure

aatma-mirror/

```
|— app/
|   |— page.tsx
|   |— chat/page.tsx
|   |— admin/page.tsx
|   |— api/
|   |   |— chat/route.ts
|   |   |— admin/generate-content/route.ts
|— components/
|   |— ChatBox.tsx
|   |— MessageBubble.tsx
|   |— VerseCard.tsx
|   |— PausePulseCard.tsx
|   |— ForwardMirrorCard.tsx
|   |— EmotionBadge.tsx
|   |— CrisisAlert.tsx
|— data/
|   |— gita-verses.json
|— lib/
|   |— ai/
```



## 8. Demo Script for Judges

Use this exact flow.

### Demo 1: Normal Query

User enters:

I am stressed about exams. I feel like I will fail.

Show:

- Emotion: Stress
  - Risk: Low
  - PausePulse
  - Verse: Chapter 2, Verse 47
  - AI response:
    - Acknowledgement
    - Reflection
    - Gita insight
    - Practical step
    - FutureMirror
- 

### Demo 2: Anger Query

User enters:

I am angry at my friend and want to insult him.

Show:

- Emotion: Anger
  - Verse about anger clouding judgment
  - AI says:
    - Pause before responding
    - Write message but do not send immediately
    - Choose calm communication
- 

### **Demo 3: Crisis Query**

User enters:

I don't want to live anymore.

Show:

- Risk: Crisis
  - Crisis Escalator activated
  - No normal spiritual advice
  - Immediate human support message
- 

### **Demo 4: Admin Content**

Admin enters:

Topic: Exam stress

Type: Reel

Show generated:

- Hook
  - Reel script
  - Caption
  - Hashtags
- 

## **9. What to Build vs What to Fake**

For hackathon, it is acceptable to mock some parts if the demo is honest.

### **Build for Real**

- Chat UI
- Emotion detection
- Crisis detection
- Gita retrieval
- AI response
- Admin content generation

### **Can Be Mocked**

- User authentication
- Subscription
- Full database
- Voice tone analysis
- Counsellor dashboard
- Analytics heatmap

- Wearable integration
- 

## 10. Final 3-Day Timeline

### Day 1 — Product Skeleton

9 AM – 11 AM: Finalize PRD + dataset

11 AM – 2 PM: Build UI screens

2 PM – 5 PM: Build chat API

5 PM – 8 PM: Add keyword verse retrieval

8 PM – 10 PM: Polish chat response UI

### Day 2 — AI Core

9 AM – 11 AM: Emotion/risk detection

11 AM – 2 PM: LLM integration

2 PM – 4 PM: Constitution prompt

4 PM – 6 PM: Crisis escalator

6 PM – 9 PM: Test demo scenarios

9 PM – 10 PM: Fix bugs

### Day 3 — Demo & Deployment

9 AM – 11 AM: Admin content generator

11 AM – 1 PM: UI polish

1 PM – 3 PM: Deploy to Vercel

3 PM – 5 PM: Record backup demo

5 PM – 7 PM: Final PPT screenshots

7 PM – 9 PM: Pitch rehearsal

---

## **11. Final Recommendation**

For 3 days, build this as:

### **A demo-first reflective AI platform**

Not a full mental health product.

Your winning angle should be:

Most AI companions react.

AatmaMirror helps students pause, reflect, and choose wisely.

Your strongest technical story:

Emotion Detection

+ PausePulse

+ Scripture RAG

+ Constitutional AI

+ Crisis Escalation

+ Admin Content Engine

Your strongest product story:

A safer alternative to generic AI companions for student mental wellness.

Your strongest demo story:

Normal stress query → reflective Gita guidance

Anger query → future consequence reflection

Crisis query → safe escalation

Admin query → reel/blog generation

That is enough to make a strong hackathon impression.